

EdgeTwin AI - Recommended Tech Stack

EdgeTwin AI - Technology Stack Document

1. Frontend

Layer	Technology	Purpose
Framework	React 18 + TypeScript	UI development, type safety
Build Tool	Vite	Fast dev/build
Styling	Tailwind CSS + Shadcn UI	Utility-first styling, prebuilt components
Routing	React Router	Multi-role dashboard navigation
Server State	React Query	API caching, sync, polling for live sensor data
Client State	Redux Toolkit	Auth, role, ticket/alert global state
Animation	Framer Motion	Transitions, alert pop-ups
Charts	Recharts	Health score, expense graphs, RUL trends
Forms	React Hook Form + Zod	Report Issue form, validation
Icons	Lucide Icons	Consistent icon set
Real-time	WebSocket / Socket.IO client	Live alerts, mechanic presence
3D / Digital Twin	Three.js / React Three Fiber	Digital twin visualization

2. Backend

Layer	Technology	Purpose
API Framework	FastAPI (Python)	REST APIs, async performance
Auth	JWT + OAuth2 (FastAPI Security)	Role-based access control
Real-time	WebSockets (FastAPI) or Socket.IO	Alerts, mechanic login status
Background Jobs	Celery + Redis	Auto-escalation timers, notifications
	SQLAlchemy /	Database access

ORM	SQLModel	layer
Validation	Pydantic	Request/response schemas
File Storage	AWS S3 / MinIO	Photo & voice note uploads

3. AI / ML & Digital Twin

Layer	Technology	Purpose
ML Framework	Scikit-learn / XGBoost	Health score, risk classification
Deep Learning (optional)	PyTorch / TensorFlow	Failure type & RUL prediction
Time-series	Prophet / LSTM	RUL forecasting
Feature Store	Feast (optional)	Reusable ML features
Model Serving	FastAPI + ONNX Runtime	Low-latency inference
Digital Twin Engine	Custom simulation service (Python)	Virtual machine state sync

4. Database & Storage

Layer	Technology	Purpose
Primary DB	PostgreSQL	Machines, tickets, users, cost logs
Time-series DB	TimescaleDB / InfluxDB	High-frequency sensor readings
Cache	Redis	Session, mechanic presence, rate limiting
Object Storage	AWS S3 / MinIO	Media attachments

5. IoT / Edge Layer

Layer	Technology	Purpose
Protocol	MQTT (Mosquitto/EMQX)	Sensor data ingestion
Edge Gateway	Node-RED / Python edge agent	Preprocessing at machine site
Streaming	Kafka (scale-up option)	High-volume sensor stream pipeline

6. DevOps & Infrastructure

Layer	Technology	Purpose
Containerization	Docker	Service packaging
Orchestration	Kubernetes	Scaling, multi-service deployment
CI/CD	GitHub Actions	Build/test/deploy pipeline

Monitoring	Grafana + Prometheus	System & sensor metrics dashboards
Logging	ELK Stack / Loki	Centralized logs, audit trail
Hosting	AWS / Azure / GCP	Cloud infrastructure

7. Notifications

Layer	Technology	Purpose
Push/Browser	Web Push API	Browser notifications
Email	SendGrid / SES	Escalation & ticket emails
SMS (optional)	Twilio	Critical alert SMS

8. Security & Compliance

- JWT-based auth with role claims (Operator, Mechanic, Maintenance Manager, Production Manager, Factory Owner, Admin)
- HTTPS/TLS everywhere
- Input validation via Pydantic + Zod (frontend/backend parity)
- Audit logs for ticket status changes and cost updates
- WCAG AA accessibility compliance on frontend

9. Summary Architecture Flow

```

IoT Sensors --MQTT--> Edge Gateway --> Kafka/Direct --> FastAPI
Ingestion
    --> TimescaleDB (raw) + PostgreSQL (aggregated)
    --> ML Inference Service --> Predictions stored
    --> WebSocket push --> React Dashboards (role-based)
    --> Celery workers handle escalation timers & notifications

```